Part 1

Testing and Development Overview

For the development of this system, a shift-left approach is recommended.

> The main idea behind Shift-Left is to start testing earlier in the process. By being proactive, you can catch problems sooner and fix them right away, which saves […] money and effort compared to finding bugs later.

(Kesavan, 2024, p.1)

Implement this by testing at all stages of an agile development process with CI/CD. Agile allows discrepancies found by tests to be factored into the plan and workload. Agile also gives flexibility to change the plan if requirements are changed, which is crucial for a system responding to natural disasters. Additionally, as an emergency response system, users will not know when the system is required, and continuous deployment allows a Minimum Viable Product to be available as soon as possible, and the most up-to-date version to always be used.

To effectively implement CI/CD, a pipeline should be set up which allows changes to be merged, then tested in various environments before being deployed. Locally, a developer should write and test changes to a small update. Then, the changes can be merged and tested on development environments. This can be one environment, or multiple environments, depending on the nature of the tests (see Automated CI/CD pipeline overview). Finally, it can be sent to production environments. Again, this can be one or multiple (see Production environment(s)).

Local Testing

In the local stage, TDD should be used, bringing testing early in the process. Writing unit and contract tests before the code is developed encourages the testing to be as extensive as possible and encourages developers to consider the testability of the code as they implement it. Contract tests are used to test API requests and responses *(Pact and Fellows, 2023)*. These can be mocked or complete (depending on whether the API call is internal or external). In addition to TDD, developers should write integration tests in order to capture the interactions between different parts of the code. The simplicity and durability of unit, contract and integration tests means they can cover large amounts of scope with minimal time and maintenance. As the application is data-based and requires accuracy, these tests are vital to ensure that small discrepancies and unexpected results don’t go unnoticed.

BDD should not be used here as it takes a lot of time and does not cover the majority of this system (as it is primarily data import and analysis). It could be useful for checking that results are returned in the correct places, but it cannot check that they are the correct results. BDD is better suited where the front-end is central to the application, and user experience can be used to evaluate the correct functionality of the system.

Automated UI tests could be used at this stage, but they are volatile and take a long time to both write and run. If components are moved, renamed or changed, they can break easily. Manual testing is better suited for this system as it is more flexible. At this stage, simple sense checks and exploratory tests can be carried out by developers on features they have changed, and wider exploratory testing can be carried out later in the process (see Staging environment).

In addition to these code tests, SAST and SCA should be run on all code before merging. These are light on resource, as pre-built SAST and SCA packages can be used and run. This may cost money depending on the frameworks and languages being used, but are important nonetheless, as security is essential to this system. It has significance to social and life-threatening issues, and high media coverage, making this a desirable system to target.

Automated CI/CD Pipeline Overview

Once changes are pushed, they should be tested on development environments. Three environments are recommended: one for lighter smoke tests which can be run on each merge but need to be tested with more reliable components than can be done locally (the “integration environment” below); one for more comprehensive testing, which can be run on groups of changes before a deployment (the “staging environment” below); one for destructive tests, which is designed to be safe to break (the “security environment” below). This allows the CI/CD process to be run continuously with comprehensive testing, without different stages affecting each other.

The integration environment can be lighter-weight, as it is not a production-like environment. The staging and security environments should be as similar to production environments as possible, with well-simulated/mocked versions of the external services. Lots of the external data sources are unpredictable and inconsistent, with periods of high and low activity. They may come with unexpected noise or badly formatted data. They are likely to have spikes at similar times to each other, as they are responsive to natural disasters. While this cannot be perfectly mocked, you should attempt to simulate this variation and inconsistency.

Integration Environment

The integration environment sits earlier in the CI/CD pipeline than the other development environments and should be used for simple smoke tests. This should include integration tests (both at the service level and the API level), unit tests and contract tests.

In a system with many interacting components, integration testing is critical for exposing issues which are not apparent during isolated testing by an individual developer pre-commit.

Security Environment

The security environment is for destructive security tests. As stated earlier, the system is a target for attack and should be thoroughly security-tested. Dynamic Application Security Testing is an easy way to test for vulnerabilities without requiring tester/developer time, as pre-built frameworks can be set up in the pipeline. It simulates malicious attacks, covering data/input vulnerability, authentication checks, and session management *(CrowdStrike, 2025)*. This is not sufficient alone but it allows human testers to focus on more complex tests.

There are many potential weak points in the system, as it links to many external services, including both front-ends, Twitter, external agencies, and other parts of the internet being scraped for data. All of these should be covered by penetration tests (where testers attempt to breach security *(National Cyber Security Centre, 2017)*). These are time-consuming, but necessary on a public-facing and critical system. To increase the effectiveness of penetration testing, Interactive Application Security Testing (IAST) can be injected into the application while the tests are being run *(Crowdstrike, 2025)*. This is a low-cost way to increase the insight gained by tests, by looking at the internal application as it is running.

Fuzzing, which runs random/generative input/output tests *(BlackDuck, c.2026)*, could also be used at this stage, but the results are often unclear and can take a lot of time for developers/testers to interpret and fix. With the above security testing in place, fuzzing is not necessary, given the time overhead.

Staging Environment

The staging environment is a production-like environment for tests. The primary focus should be load testing. The application will have a high load on all of its endpoints, so all of these should be tested, both isolated and as a whole system. Benchmarks should be set based on expected traffic, which can be discussed with local authorities. This testing should also ensure that denial of service attacks can be flagged, and that mitigations for them are sufficient.

In addition to load testing, the user interfaces should be tested at this stage. In this case, exploratory testing is more appropriate than UAT, as the product owner is not the end user. Both UIs should be tested while attempting to simulate a high-stress environment, with different focuses. The call-centre site should allow users to quickly retrieve and convey information with a high volume of calls, and the public-facing site should be easy to navigate and prioritise showing easily digested advice over more complex details.

Production Environment(s)

Finally, the application can be deployed. Beta testing versions could be released at this stage, or A/B tests could be used, but they are not necessary for this system. The focus of the application is to be functional and resilient in times of emergency, which is covered by the testing outlined above. While beta or A/B testing could find some usability issues, their primary function is to identify user preferences and refine UX. It would take lots of time and resource to find users, gather data, allow them to use the system, and respond to their results. It would not be worth the cost and it is more important that this system has reliable versions released quickly than having extra UI/usability testing. It is also important that advice being given is consistent, which could not be guaranteed with multiple versions in use at the same time. The agile process involves regular communication with the product owner, so most issues which could have been found in these stages of testing can be reported back and resolved this way instead.

References

Black Duck. “Glossary: Fuzz Testing.” *Black Duck, c. 2026*, https://www.blackduck.com/glossary/what-is-fuzz-testing.html.

Gale, Jamie. “Dynamic Application Security Testing (DAST) Explained.” *CrowdStrike, 15-04-2025*, https://www.crowdstrike.com/en-us/cybersecurity-101/cloud-security/dynamic-application-security-testing-dast/.

Gale, Jamie. “Introduction to Interactive Application Security Testing (IAST).” *CrowdStrike, 10-04-2025*, https://www.crowdstrike.com/en-us/cybersecurity-101/cloud-security/interactive-application-security-testing-iast/.

Kesavan, Elavarasi. “Shift-Left and Continuous Testing in Quality Assurance Engineering Ops and DevOps.” *International Journal of Scientific Research and Modern Technology, vol. 3, no. 1, p. 1, 2024*, https://www.researchgate.net/publication/396863242_Shift-Left_and_Continuous_Testing_in_Quality_Assurance_Engineering_Ops_and_DevOps.

National Cyber Security Centre. “Penetration testing.” *National Cyber Security Centre, 08-08-2017*, https://www.ncsc.gov.uk/guidance/penetration-testing.

Fellows, Matt. “What is contract testing and why should I try it?” *Pact Flow, 02-09-2023*, https://pactflow.io/blog/what-is-contract-testing/.

Part 2

Assumptions

- Wind speeds can only be integers: it is not type-hinted, but the range() function is used.
- I do not need to test for negative (or other unreasonable) inputs: Not explicitly specified in the brief or attempted anywhere in the code.
- I do not need to type check (except for a dictionary in update_storm): There is very little type hinting or attempt to type check throughout the code and I assume this is not intended to be a large repeat of the same bug.
- update_storm is for updating *temperature* and *windspeed*: Both of these might change, but *name* shouldn't

Initial bug fixes

I think the below errors were just my linter being strict so I made some initial changes to prevent this from being an issue before starting testing, which I have outlined below.

Type hinting Storm

```
    @abstractmethod
    def calculate_classification(self) -> str: # type hint
        pass

    @abstractmethod
    def get_advice(self) -> str: #type hint
        pass
```

Replacing 'or' with '|'

```
def view_storm(self, name: str) -> Storm | None: #replace "or" with "|"
        for storm in self.storm_list:
            if storm.name == name:
                return storm
        return None
```

Hurricane

Fail 1

```
=================================== FAILURES ===================================
_____________________________ test_create_tropical_storm _____________________________

    def test_create_tropical_storm():
        hurricane = Hurricane("Amy", 70)
>       assert hurricane.calculate_classification() == "Tropical Storm"
E       AssertionError: assert 'Unclassified' == 'Tropical Storm'
E
E         - Tropical Storm
E         + Unclassified

tests\test_hurricane.py:6: AssertionError
```

```
def test_create_tropical_storm():
    hurricane = Hurricane("Amy", 70)
    assert hurricane.calculate_classification() == "Tropical Storm"
    assert hurricane.get_advice() == "Close storm shutters and stay away from windows"
```

Issue: Identifying Tropical Storms as "Unclassified"

Fix: Set default to Tropical Storm

```
def calculate_classification(self) -> str:
        if self.wind_speed in range(74, 95):
            return "Category one"
        elif self.wind_speed in range(96, 111):
            return "Category two"
        elif self.wind_speed in range(111, 130):
            return "Category three"
        elif self.wind_speed in range(130, 157):
            return "Category four"
        elif self.wind_speed > 156:
            return "Category five"
        return "Tropical Storm" # replaced "Unclassified" with "Tropical Storm"
```

```
tests/test_hurricane.py::test_create_tropical_storm PASSED
```

Fail 2

```
_____ test_create_category_one_UB_in _____

    def test_create_category_one_UB_in():
        hurricane = Hurricane("Bram", 95)
>       assert hurricane.calculate_classification() == "Category one"
E       AssertionError: assert 'Tropical Storm' == 'Category one'
E
E         - Category one
E         + Tropical Storm

tests\test_hurricane.py:26: AssertionError
```

```
def test_create_category_one_UB_in():
    hurricane = Hurricane("Bram", 95)
    assert hurricane.calculate_classification() == "Category one"
    assert hurricane.get_advice() == "Close storm shutters and stay away from windows"
```

Issue: Upper bound for category 1 not inside bound

Fix: Use exclusive upper bound

```
    def calculate_classification(self) -> str:
        if self.wind_speed in range(74, 96): # replaced 95 with 96
            return "Category one"
```

```
        elif self.wind_speed in range(96, 111):
            return "Category two"
        elif self.wind_speed in range(111, 130):
            return "Category three"
        elif self.wind_speed in range(130, 157):
            return "Category four"
        elif self.wind_speed > 156:
            return "Category five"
        return "Tropical Storm" # replaced "Unclassified" with "Tropical Storm"
```

```
tests/test_hurricane.py::test_create_category_one_UB_in PASSED
```

Tornado

Fail 1

```
_____ test_create_f0_LB_in _____

    def test_create_f0_LB_in():
        tornado = Tornado("Gerard", 40)
>       assert tornado.calculate_classification() == "F0"
E       AssertionError: assert 'Unclassified' == 'F0'
E
E         - F0
E         + Unclassified

tests\test_tornado.py:11: AssertionError
```

```
def test_create_f0_LB_in():
    tornado = Tornado("Gerard", 40)
    assert tornado.calculate_classification() == "F0"
    assert tornado.get_advice() == "Find safe room/shelter, shield yourself from debris"
```

Issue: Lower bound for f0 not inside bound

Fix: Use less than

```
def calculate_classification(self) -> str:
        if self.wind_speed < 40: # changed <= to <
            return "Unclassified"
        elif self.wind_speed <= 72:
            return "F0"
        elif self.wind_speed in range(73, 113):
            return "F1"
        elif self.wind_speed in range(113, 158):
            return "F2"
        elif self.wind_speed in range(158, 206):
            return "F3"
        elif self.wind_speed in range(206, 261):
            return "F4"
        elif self.wind_speed >= 261:
            return "F5"
        return "Unclassified"
```

```
tests/test_tornado.py::test_create_f0_LB_in PASSED
```

Fail 2

```
_____ test_create_f4 _____

    def test_create_f4():
        tornado = Tornado("Kasia", 250)
        assert tornado.calculate_classification() == "F4"
>       assert tornado.get_advice() == "Find underground shelter, crouch near to the floor covering your head with your hands"
E       AssertionError: assert 'There is no need to panic' == 'Find undergr...th your hands'
E
E         - Find underground shelter, crouch near to the floor covering your head with your hands
E         + There is no need to panic

tests\test_tornado.py:67: AssertionError
```

```python
def test_create_f4():
    tornado = Tornado("Kasia", 250)
    assert tornado.calculate_classification() == "F4"
    assert tornado.get_advice() == "Find underground shelter, crouch near to the floor covering your head with your hands"
```

Issue: F4 in wrong group

Fix: Add comma

```python
def get_advice(self) -> str:
        classification = self.calculate_classification()

        if classification in ["Unclassified", "F0", "F1"]:
            return "Find safe room/shelter, shield yourself from debris"
        elif classification in ["F2", "F3", "F4", "F5"]: # added comma between "F4" and "F5"
            return "Find underground shelter, crouch near to the floor covering your head with your hands"
        return "There is no need to panic"
```

```
tests/test_tornado.py::test_create_f4 PASSED
```

Fail 3

```
_____ test_create_unclassified_tornado _____

    def test_create_unclassified_tornado():
        tornado = Tornado("Marty", 0)
        assert tornado.calculate_classification() == "Unclassified"
>       assert tornado.get_advice() == "There is no need to panic"
E       AssertionError: assert 'Find safe ro...f from debris' == 'There is no need to panic'
E
E         - There is no need to panic
E         + Find safe room/shelter, shield yourself from debris

tests\test_tornado.py:92: AssertionError
```

```python
def test_create_unclassified_tornado():
    tornado = Tornado("Marty", 0)
    assert tornado.calculate_classification() == "Unclassified"
    assert tornado.get_advice() == "There is no need to panic"
```

Issue: Unclassified in wrong group

Fix: Remove "Unclassified" from wrong group

```
    def get_advice(self) -> str:
        classification = self.calculate_classification()

        if classification in ["F0", "F1"]: # removed "Unclassified"
            return "Find safe room/shelter, shield yourself from debris"
        elif classification in ["F2", "F3", "F4", "F5"]: # added comma between "F4" and "F5"
            return "Find underground shelter, crouch near to the floor covering your head with your
    hands"
        return "There is no need to panic"
```

```
tests/test_tornado.py::test_create_unclassified_tornado PASSED
```

Blizzard

Fail 1

```
_____ test_create_blizzard _____

    def test_create_blizzard():
        blizzard = Blizzard("Oscar", 40, 10)
>       assert blizzard.calculate_classification() == "Blizzard"
E       AssertionError: assert 'blizzard' == 'Blizzard'
E
E         - Blizzard
E         ? ^
E         + blizzard
E         ? ^

tests\test_blizzard.py:21: AssertionError
```

```
    def test_create_blizzard():
        blizzard = Blizzard("Oscar", 40, 10)
        assert blizzard.calculate_classification() == "Blizzard"
        assert blizzard.get_advice() == "Keep warm, Do not travel unless absolutely essential."
```

Issue: "Blizzard" is not capitalised

Fix: Capitalise it

```
    def calculate_classification(self) -> str:
        if self.wind_speed >= 35:
            return "Blizzard" # capitalised "blizzard"
        elif self.wind_speed >= 45 and self.temp <= -12:
            return "Severe Blizzard"
        return "Snow Storm"
```

```
_____ test_create_blizzard _____

    def test_create_blizzard():
        blizzard = Blizzard("Oscar", 40, 10)
        assert blizzard.calculate_classification() == "Blizzard"
>       assert blizzard.get_advice() == "Keep warm, Do not travel unless absolutely essential."
E       AssertionError: assert 'Take care an... if possible.' == 'Keep warm, D...ly essential.'
E
E         - Keep warm, Do not travel unless absolutely essential.
E         + Take care and avoid travel if possible.

tests\test_blizzard.py:22: AssertionError
```

Initial issue was fixed.

New issue: Blizzard in wrong category

Fix: Create category for blizzard

```python
    def get_advice(self) -> str:
        classification = self.calculate_classification()

        if classification == "Severe Blizzard":
            return "Keep warm, avoid all travel."
        # inserted below condition for Blizzard
        elif classification == "Blizzard":
            return "Keep warm, Do not travel unless absolutely essential."
        return "Take care and avoid travel if possible."
```

```
tests/test_blizzard.py::test_create_blizzard PASSED
```

Fail 2

```
_____ test_create_severe_blizzard _____

    def test_create_severe_blizzard():
        blizzard = Blizzard("Patrick", 400, -100)
>       assert blizzard.calculate_classification() == "Severe Blizzard"
E       AssertionError: assert 'Blizzard' == 'Severe Blizzard'
E
E         - Severe Blizzard
E         + Blizzard

tests\test_blizzard.py:51: AssertionError
```

```python
    def test_create_severe_blizzard():
        blizzard = Blizzard("Patrick", 400, -100)
        assert blizzard.calculate_classification() == "Severe Blizzard"
        assert blizzard.get_advice() == "Keep warm, avoid all travel."
```

Issue: Severe Blizzard not being identified

Fix: Check for Severe Blizzard before Blizzard

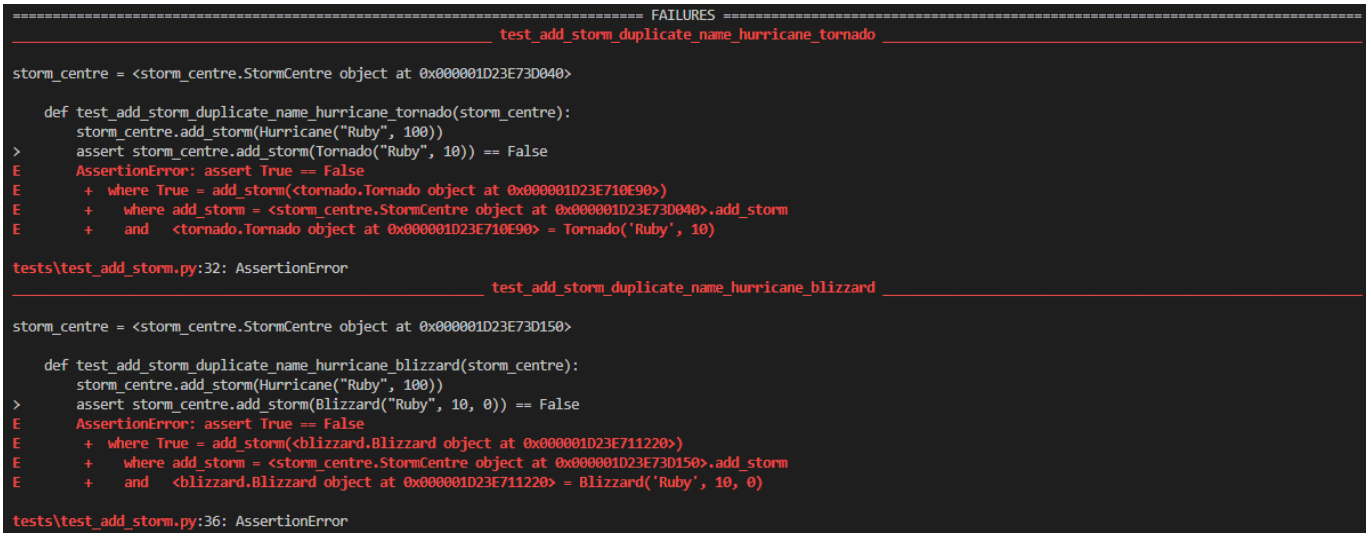
```python
    def calculate_classification(self) -> str:
        if self.wind_speed >= 45 and self.temp <= -12: # moved this condition to top
            return "Severe Blizzard"
        elif self.wind_speed >= 35:
```

```
            return "Blizzard" # capitalised "blizzard"
        return "Snow Storm"
```

```
tests/test_blizzard.py::test_create_severe_blizzard PASSED
```

Add storm

Fail 1




```
    def test_add_storm_duplicate_name_different_type(storm_centre):
        storm_centre.add_storm(Hurricane("Ruby", 100))
        assert storm_centre.add_storm(Tornado("Ruby", 10)) == False
```

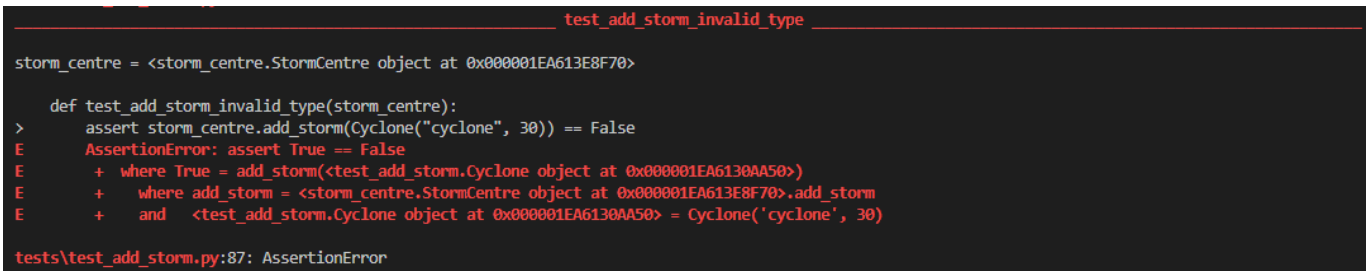
Issue: Duplicate name allowed when one of the types is a Hurricane

Fix: Correctly assign name in Hurricane

```
    def __init__(self, name, wind_speed):
            super().__init__(name, wind_speed) # replaced "none" with name
```

```
tests/test_add_storm.py::test_add_storm_duplicate_name_hurricane_tornado PASSED
tests/test_add_storm.py::test_add_storm_duplicate_name_hurricane_blizzard PASSED
```

Fail 2



```
class Cyclone (Storm):
    def __init__(self, name, wind_speed):
        super().__init__(name, wind_speed)

    def calculate_classification(self) -> str: # type hint
        return "Tropical"

    def get_advice(self) -> str: #type hint
        return "RUN"

def test_add_storm_invalid_type(storm_centre):
    assert storm_centre.add_storm(Cyclone("cyclone", 30)) == False
```

Issue: Storms which are not a Blizzard, Tornado or Hurricane can be added

Fix: Fix condition

```
def add_storm(self, storm: Storm) -> bool:
    if (len(self.storm_list) <= 20
            and (isinstance(storm, Blizzard) or isinstance(storm, Tornado) or isinstance(storm,
Hurricane)) # fixed this check
            and not self.already_exists(storm.name)): # refactored this line for ease of
reading
        self.storm_list.append(storm)
        return True
    return False
```

```
tests/test_add_storm.py::test_add_storm_invalid_type PASSED
```

Fail 3

```
_____ test_add_storm_21 _____

storm_centre = <storm_centre.StormCentre object at 0x0000029506167590>

    def test_add_storm_21(storm_centre):
        for i in range(20):
            assert storm_centre.add_storm(Hurricane(str(i), 100)) == True
>       assert storm_centre.add_storm(Hurricane("21", 100)) == False
E       AssertionError: assert True == False
E        +  where True = add_storm(<hurricane.Hurricane object at 0x00000295063F4EF0>)
E        +    where add_storm = <storm_centre.StormCentre object at 0x0000029506167590>.add_storm
E        +    and   <hurricane.Hurricane object at 0x00000295063F4EF0> = Hurricane('21', 100)

tests\test_add_storm.py:73: AssertionError
```

```
def test_add_storm_21(storm_centre):
    for i in range(20):
        assert storm_centre.add_storm(Hurricane(str(i), 100)) == True
    assert storm_centre.add_storm(Hurricane("21", 100)) == False
```

Issue: More than 20 storms can be added

Fix: Ensure less than 20 storms are in centre before adding

```
    def add_storm(self, storm: Storm) -> bool:
        if (len(self.storm_list) < 20 # changed <= to <
                and (isinstance(storm, Blizzard) or isinstance(storm, Tornado) or isinstance(storm,
    Hurricane))
                and not self.already_exists(storm.name)):
            self.storm_list.append(storm)
            return True
        return False
```

```
tests/test_add_storm.py::test_add_storm_21 PASSED
```

Remove storm

Fail 1

```
_____ test_remove_storm_doesnt_exist _____

full_storm_centre = <storm_centre.StormCentre object at 0x000001AF2DD0D9B0>

    def test_remove_storm_doesnt_exist(full_storm_centre):
>       assert full_storm_centre.remove_storm("hello") == False
E       AssertionError: assert True == False
E        +  where True = remove_storm('hello')
E        +    where remove_storm = <storm_centre.StormCentre object at 0x000001AF2DD0D9B0>.remove_storm

tests\test_remove_storm.py:16: AssertionError
```

```
    def test_remove_storm_doesnt_exist(full_storm_centre):
        assert full_storm_centre.remove_storm("hello") == False
```

Issue: Does not indicate if removed storm not found

Fix: Make return value conditional

```
    def remove_storm(self, name: str) -> bool:
        for storm in self.storm_list:
            if name == storm.name:
                self.storm_list.remove(storm)
                return True # made this conditional
        return False # added default false return
```

```
tests/test_remove_storm.py::test_remove_storm_doesnt_exist PASSED
```

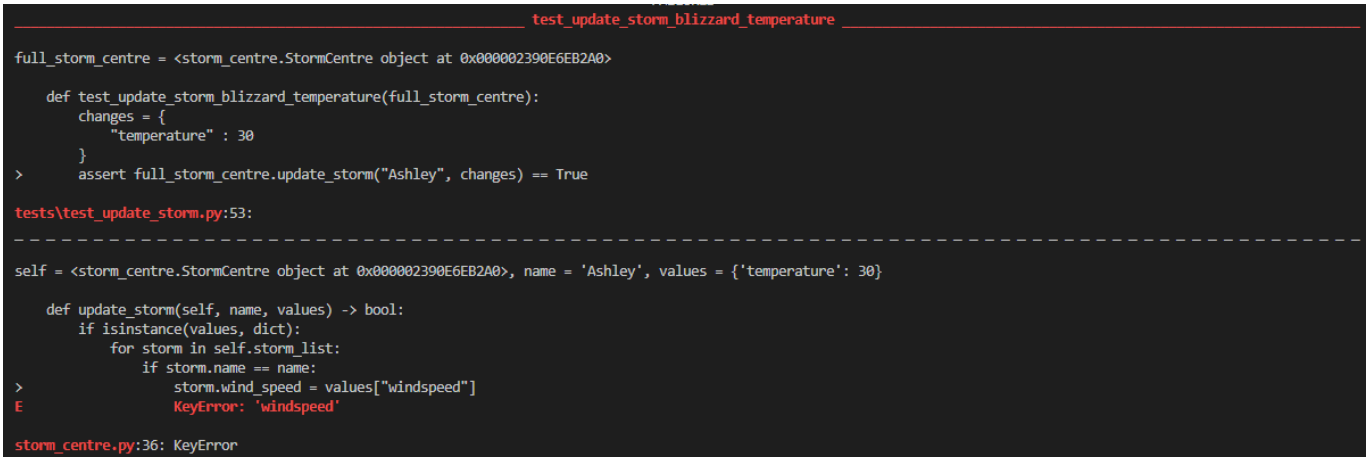
Update storm

I have created a helper function to test whether storms are equivalent

```
    def _equivalent(actual: Storm, expected: Storm) -> bool:
        if type(actual) != type (expected):
            return False
        if type(actual) == Blizzard and type(expected) == Blizzard:
```

```
            if (actual.wind_speed == expected.wind_speed
                and actual.temp == expected.temp
                and actual.name == expected.name):
                return True
            return False
        else:
            if (actual.wind_speed == expected.wind_speed
                and actual.name == expected.name):
                return True
            return False
```

Fail 1




```
    def test_update_storm_blizzard_temperature(full_storm_centre):
        changes = {
            "temperature" : 30
        }
        assert full_storm_centre.update_storm("Ashley", changes) == True
        actual = full_storm_centre.view_storm("Ashley")
        expected = Blizzard("Ashley", 25, 30)
        assert _equivalent(actual, expected) == True
```

Issue: Update method using windspeed when it is not given

Fix: Make update conditional

```python
def update_storm(self, name, values) -> bool:
    if isinstance(values, dict):
        for storm in self.storm_list:
            if storm.name == name:
                if ("windspeed" in values): # added condition
                    storm.wind_speed = values["windspeed"]
                    return True
    else:
        raise Exception("Values must be provided as a dictionary")
    return False
```

```
_____ test_update_storm_blizzard_temperature _____

full_storm_centre = <storm_centre.StormCentre object at 0x000002573809B2A0>

    def test_update_storm_blizzard_temperature(full_storm_centre):
        changes = {
            "temperature" : 30
        }
>       assert full_storm_centre.update_storm("Ashley", changes) == True
E       AssertionError: assert False == True
E        +  where False = update_storm('Ashley', {'temperature': 30})
E        +    where update_storm = <storm_centre.StormCentre object at 0x000002573809B2A0>.update_storm

tests\test_update_storm.py:53: AssertionError
```

Initial issue was fixed.

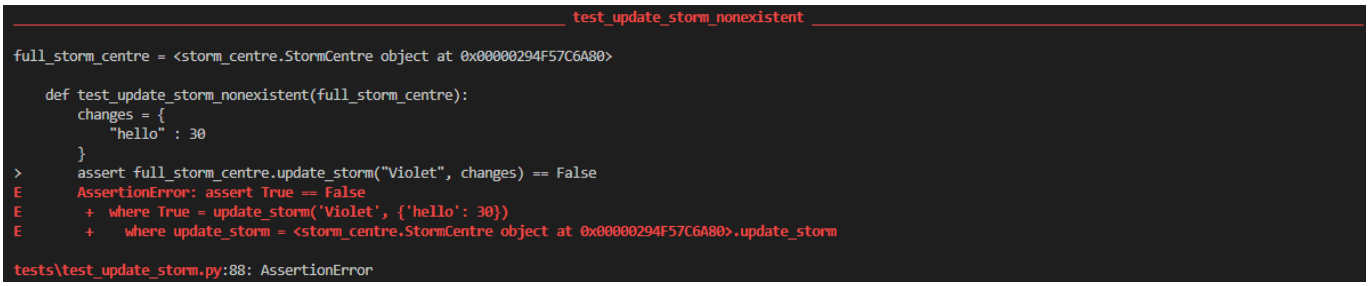
New issue: Updating the temperature does not work

Fix: Add condition to update temperature

```python
def update_storm(self, name, values) -> bool:
    if isinstance(values, dict):
        for storm in self.storm_list:
            if storm.name == name:
                if ("temperature" in values): # added condition for temperature
                    if isinstance(storm, Blizzard):
                        storm.temp = values["temperature"]
                    else:
                        return False
                if ("windspeed" in values): # added condition
                    storm.wind_speed = values["windspeed"]
    else:
        raise Exception("Values must be provided as a dictionary")
    return True # replaced default with True and use False returns for error
```

```
tests/test_update_storm.py::test_update_storm_blizzard_temperature PASSED
```

Fail 2

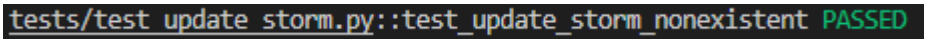



```
def test_update_storm_nonexistent(full_storm_centre):
    changes = {
        "hello" : 30
    }
    assert full_storm_centre.update_storm("Violet", changes) == False
    actual = full_storm_centre.view_storm("Violet")
    expected = Hurricane("Violet", 25)
    assert _equivalent(actual, expected) == True
```

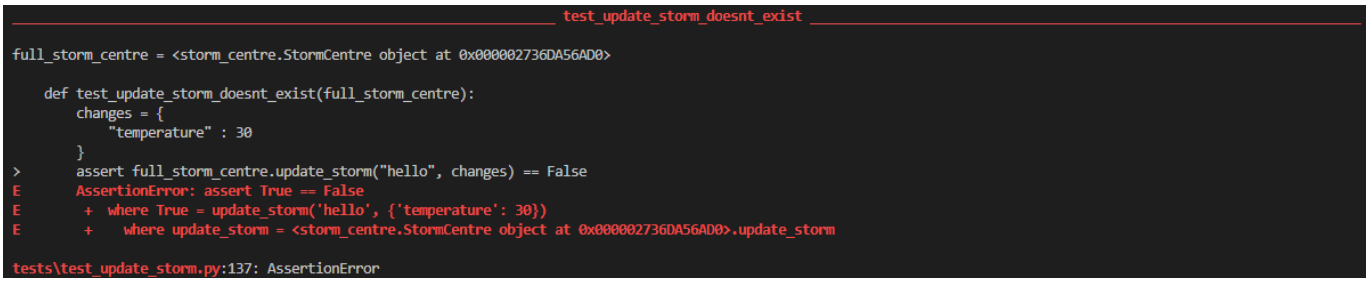
Issue: Update returning true when key is invalid

Fix: Add check for valid keys only

```
def update_storm(self, name, values) -> bool:
    if isinstance(values, dict):
        if (not (set(values.keys()).issubset({"temperature", "windspeed"}))): # added check for
valid keys
            return False
        for storm in self.storm_list:
            if storm.name == name:
                if ("temperature" in values): # added condition for temperature
                    if isinstance(storm, Blizzard):
                        storm.temp = values["temperature"]
                    else:
                        return False
                if ("windspeed" in values): # added condition
                    storm.wind_speed = values["windspeed"]
    else:
        raise Exception("Values must be provided as a dictionary")
    return True # replaced default with True and use False returns for errors
```



Fail 3

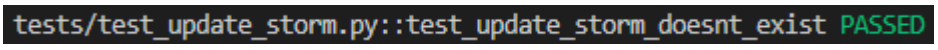



```
def test_update_storm_doesnt_exist(full_storm_centre):
    changes = {
        "temperature" : 30
    }
    assert full_storm_centre.update_storm("hello", changes) == False
    actual_violet = full_storm_centre.view_storm("Violet")
    actual_wubbo = full_storm_centre.view_storm("Wubbo")
    actual_ashley = full_storm_centre.view_storm("Ashley")
    expected = [Hurricane("Violet", 25), Tornado("Wubbo", 25), Blizzard("Ashley", 25, -12)]
    assert _equivalent(actual_violet, expected[0]) == True
    assert _equivalent(actual_wubbo, expected[1]) == True
    assert _equivalent(actual_ashley, expected[2]) == True
```

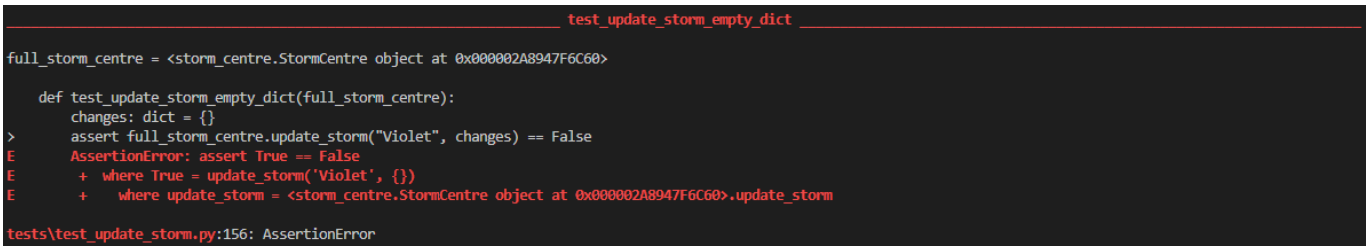
Issue: Not returning False when updating storm which doesn't exist

Fix: Check storm exists before updating

```
def update_storm(self, name, values) -> bool:
    if isinstance(values, dict):
        if (not self.already_exists(name)): # added check for storm existence
            return False
        if (not (set(values.keys()).issubset({"temperature", "windspeed"}))): # added check for
valid keys
            return False
        for storm in self.storm_list:
            if storm.name == name:
                if ("temperature" in values): # added condition for temperature
                    if isinstance(storm, Blizzard):
                        storm.temp = values["temperature"]
                    else:
                        return False
                if ("windspeed" in values): # added condition
                    storm.wind_speed = values["windspeed"]
    else:
        raise Exception("Values must be provided as a dictionary")
    return True # replaced default with True and use False returns for errors
```



Fail 4

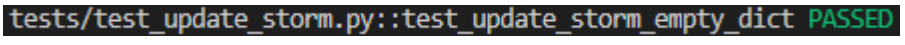



```python
def test_update_storm_empty_dict(full_storm_centre):
    changes: dict = {}
    assert full_storm_centre.update_storm("Violet", changes) == False
    actual = full_storm_centre.view_storm("Violet")
    expected = Hurricane("Violet", 25)
    assert _equivalent(actual, expected) == True
```

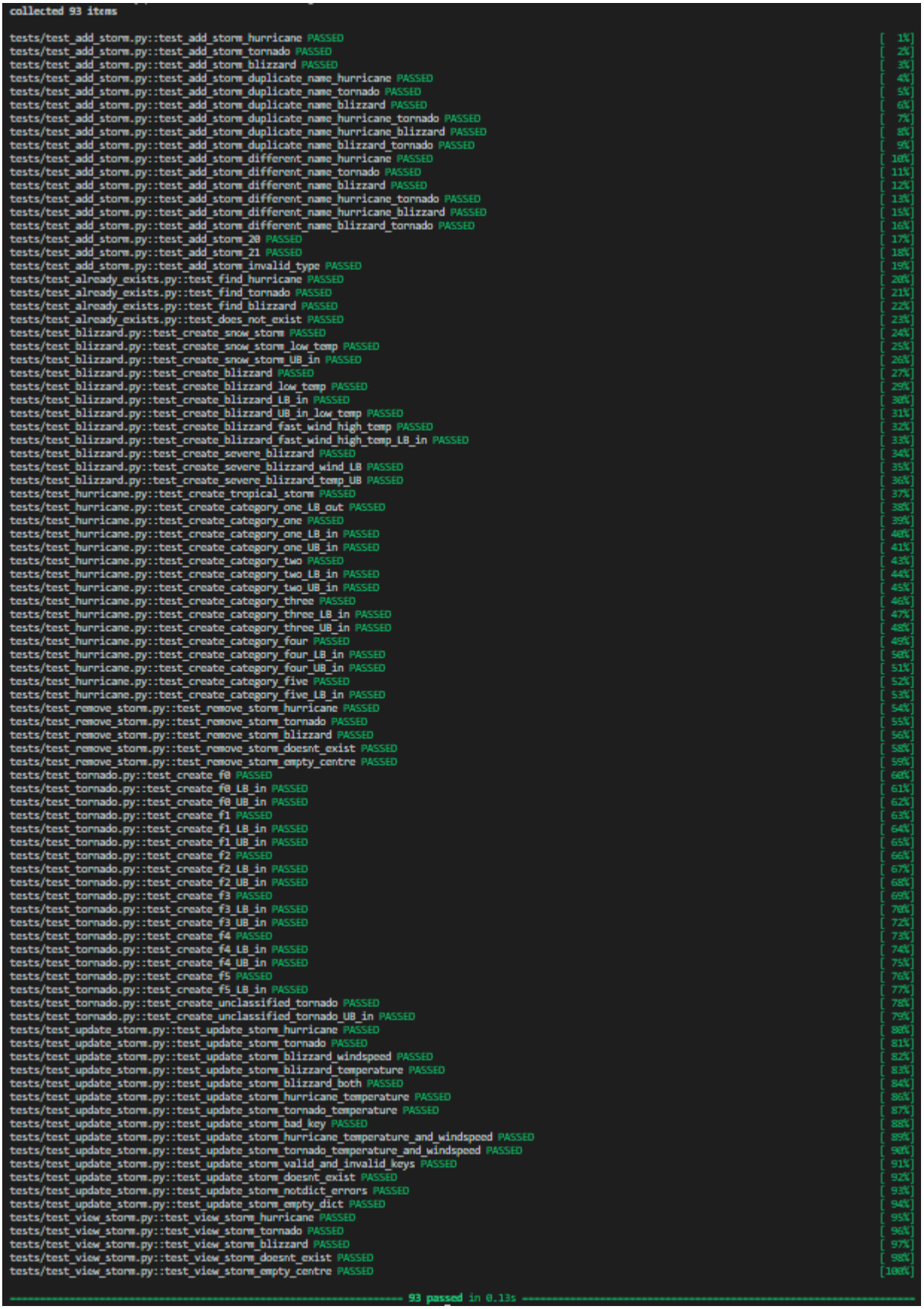
Issue: Update with an empty dict returns True

Fix: Add check for empty dict

```python
def update_storm(self, name, values) -> bool:
    if isinstance(values, dict):
        if (not self.already_exists(name)): # added check for storm existence
            return False
        if (not (set(values.keys()).issubset({"temperature", "windspeed"})) # added check for
valid keys
            or len(values.keys()) == 0): # added check for empty dict
            return False
        for storm in self.storm_list:
            if storm.name == name:
                if ("temperature" in values): # added condition for temperature
                    if isinstance(storm, Blizzard):
                        storm.temp = values["temperature"]
                    else:
                        return False
                if ("windspeed" in values): # added condition
                    storm.wind_speed = values["windspeed"]
    else:
        raise Exception("Values must be provided as a dictionary")
    return True # replaced default with True and use False returns for errors
```



All tests


collected 93 items
tests/test_add_storm.py::test_add_storm_hurricane PASSED [1%]
tests/test_add_storm.py::test_add_storm_tornado PASSED [2%]
tests/test_add_storm.py::test_add_storm_blizzard PASSED [3%]
tests/test_add_storm.py::test_add_storm_duplicate_name_hurricane PASSED [4%]
tests/test_add_storm.py::test_add_storm_duplicate_name_tornado PASSED [5%]
tests/test_add_storm.py::test_add_storm_duplicate_name_blizzard PASSED [6%]
tests/test_add_storm.py::test_add_storm_duplicate_name_hurricane_tornado PASSED [7%]
tests/test_add_storm.py::test_add_storm_duplicate_name_hurricane_blizzard PASSED [8%]
tests/test_add_storm.py::test_add_storm_duplicate_name_blizzard_tornado PASSED [9%]
tests/test_add_storm.py::test_add_storm_different_name_hurricane PASSED [10%]
tests/test_add_storm.py::test_add_storm_different_name_tornado PASSED [11%]
tests/test_add_storm.py::test_add_storm_different_name_blizzard PASSED [12%]
tests/test_add_storm.py::test_add_storm_different_name_hurricane_tornado PASSED [13%]
tests/test_add_storm.py::test_add_storm_different_name_hurricane_blizzard PASSED [15%]
tests/test_add_storm.py::test_add_storm_different_name_blizzard_tornado PASSED [16%]
tests/test_add_storm.py::test_add_storm_20 PASSED [17%]
tests/test_add_storm.py::test_add_storm_21 PASSED [18%]
tests/test_add_storm.py::test_add_storm_invalid_type PASSED [19%]
tests/test_already_exists.py::test_find_hurricane PASSED [20%]
tests/test_already_exists.py::test_find_tornado PASSED [21%]
tests/test_already_exists.py::test_find_blizzard PASSED [22%]
tests/test_already_exists.py::test_does_not_exist PASSED [23%]
tests/test_blizzard.py::test_create_snow_storm PASSED [24%]
tests/test_blizzard.py::test_create_snow_storm_low_temp PASSED [25%]
tests/test_blizzard.py::test_create_snow_storm_UB_in PASSED [26%]
tests/test_blizzard.py::test_create_blizzard PASSED [27%]
tests/test_blizzard.py::test_create_blizzard_low_temp PASSED [29%]
tests/test_blizzard.py::test_create_blizzard_LB_in PASSED [30%]
tests/test_blizzard.py::test_create_blizzard_UB_in_low_temp PASSED [31%]
tests/test_blizzard.py::test_create_blizzard_fast_wind_high_temp PASSED [32%]
tests/test_blizzard.py::test_create_blizzard_fast_wind_high_temp_LB_in PASSED [33%]
tests/test_blizzard.py::test_create_severe_blizzard PASSED [34%]
tests/test_blizzard.py::test_create_severe_blizzard_wind_LB PASSED [35%]
tests/test_blizzard.py::test_create_severe_blizzard_temp_UB PASSED [36%]
tests/test_hurricane.py::test_create_tropical_storm PASSED [37%]
tests/test_hurricane.py::test_create_category_one_LB_out PASSED [38%]
tests/test_hurricane.py::test_create_category_one PASSED [39%]
tests/test_hurricane.py::test_create_category_one_LB_in PASSED [40%]
tests/test_hurricane.py::test_create_category_one_UB_in PASSED [41%]
tests/test_hurricane.py::test_create_category_two PASSED [43%]
tests/test_hurricane.py::test_create_category_two_LB_in PASSED [44%]
tests/test_hurricane.py::test_create_category_two_UB_in PASSED [45%]
tests/test_hurricane.py::test_create_category_three PASSED [46%]
tests/test_hurricane.py::test_create_category_three_LB_in PASSED [47%]
tests/test_hurricane.py::test_create_category_three_UB_in PASSED [48%]
tests/test_hurricane.py::test_create_category_four PASSED [49%]
tests/test_hurricane.py::test_create_category_four_LB_in PASSED [50%]
tests/test_hurricane.py::test_create_category_four_UB_in PASSED [51%]
tests/test_hurricane.py::test_create_category_five PASSED [52%]
tests/test_hurricane.py::test_create_category_five_LB_in PASSED [53%]
tests/test_remove_storm.py::test_remove_storm_hurricane PASSED [54%]
tests/test_remove_storm.py::test_remove_storm_tornado PASSED [55%]
tests/test_remove_storm.py::test_remove_storm_blizzard PASSED [56%]
tests/test_remove_storm.py::test_remove_storm_doesnt_exist PASSED [58%]
tests/test_remove_storm.py::test_remove_storm_empty_centre PASSED [59%]
tests/test_tornado.py::test_create_f0 PASSED [60%]
tests/test_tornado.py::test_create_f0_LB_in PASSED [61%]
tests/test_tornado.py::test_create_f0_UB_in PASSED [62%]
tests/test_tornado.py::test_create_f1 PASSED [63%]
tests/test_tornado.py::test_create_f1_LB_in PASSED [64%]
tests/test_tornado.py::test_create_f1_UB_in PASSED [65%]
tests/test_tornado.py::test_create_f2 PASSED [66%]
tests/test_tornado.py::test_create_f2_LB_in PASSED [67%]
tests/test_tornado.py::test_create_f2_UB_in PASSED [68%]
tests/test_tornado.py::test_create_f3 PASSED [69%]
tests/test_tornado.py::test_create_f3_LB_in PASSED [70%]
tests/test_tornado.py::test_create_f3_UB_in PASSED [72%]
tests/test_tornado.py::test_create_f4 PASSED [73%]
tests/test_tornado.py::test_create_f4_LB_in PASSED [74%]
tests/test_tornado.py::test_create_f4_UB_in PASSED [75%]
tests/test_tornado.py::test_create_f5 PASSED [76%]
tests/test_tornado.py::test_create_f5_LB_in PASSED [77%]
tests/test_tornado.py::test_create_unclassified_tornado PASSED [78%]
tests/test_tornado.py::test_create_unclassified_tornado_UB_in PASSED [79%]
tests/test_update_storm.py::test_update_storm_hurricane PASSED [80%]
tests/test_update_storm.py::test_update_storm_tornado PASSED [81%]
tests/test_update_storm.py::test_update_storm_blizzard_windspeed PASSED [82%]
tests/test_update_storm.py::test_update_storm_blizzard_temperature PASSED [83%]
tests/test_update_storm.py::test_update_storm_blizzard_both PASSED [84%]
tests/test_update_storm.py::test_update_storm_hurricane_temperature PASSED [86%]
tests/test_update_storm.py::test_update_storm_tornado_temperature PASSED [87%]
tests/test_update_storm.py::test_update_storm_bad_key PASSED [88%]
tests/test_update_storm.py::test_update_storm_hurricane_temperature_and_windspeed PASSED [89%]
tests/test_update_storm.py::test_update_storm_tornado_temperature_and_windspeed PASSED [90%]
tests/test_update_storm.py::test_update_storm_valid_and_invalid_keys PASSED [91%]
tests/test_update_storm.py::test_update_storm_doesnt_exist PASSED [92%]
tests/test_update_storm.py::test_update_storm_notdict_errors PASSED [93%]
tests/test_update_storm.py::test_update_storm_empty_dict PASSED [94%]
tests/test_view_storm.py::test_view_storm_hurricane PASSED [95%]
tests/test_view_storm.py::test_view_storm_tornado PASSED [96%]
tests/test_view_storm.py::test_view_storm_blizzard PASSED [97%]
tests/test_view_storm.py::test_view_storm_doesnt_exist PASSED [98%]
tests/test_view_storm.py::test_view_storm_empty_centre PASSED [100%]
93 passed in 0.13s